

École Nationale des Sciences Appliquées — Tétouan

Année Académique 2025–2026

Projet Web Scraping

Scraping avec Python (bibliothèques) et automatisation avec n8n

Cible • Implémentation • Extraction

Étudiants : Abdessamad Ait Brahim
Ayyoub Amazal
Yassir Lagnaoui

Encadrant : M. Imad Sassi

Module : Analyse de Web

Filière : Big Data and AI

Date : Avril 2026

Table des matières

1 Introduction **2**

2 Exercice 1 : Scraping Yelp avec Python & Selenium **2**
- 2.1 Présentation et Objectif . . . 2
- 2.2 Environnement Technique . . . 2
- 2.3 Étapes Réalisées . . . 3
  - 2.3.1 Étape A : Collecte des 500 meilleurs restaurants . . . 3
  - 2.3.2 Étape B : Extraction des Reviewers . . . 3
  - 2.3.3 Étape C : Extraction de la Nationalité . . . 4
  - 2.3.4 Étape D : Classement des Nationalités . . . 4
- 2.4 Système de Checkpoint . . . 4
- 2.5 Résultats Obtenus . . . 5
- 2.6 Difficultés Rencontrées et Solutions . . . 7

3 Exercice 2 : Scraping avec Workflow Automatisé avec n8n **8**
- 3.1 Présentation et Objectif . . . 8
- 3.2 Environnement Technique . . . 8
- 3.3 Sites e-commerce ciblés . . . 8
- 3.4 Étapes Réalisées . . . 8
  - 3.4.1 Étape A : Installation de n8n avec Docker . . . 8
  - 3.4.2 Étape B : Accès à l'interface n8n . . . 9
  - 3.4.3 Étape C : Identification des sélecteurs CSS . . . 9
  - 3.4.4 Étape D : Création du Workflow n8n . . . 9
  - 3.4.5 Étape E : Configuration des nœuds HTTP Request . . . 10
  - 3.4.6 Étape F : Configuration des nœuds HTML Extract . . . 10
  - 3.4.7 Étape G : Configuration des nœuds Set . . . 11
  - 3.4.8 Étape H : Configuration du nœud Merge . . . 12
  - 3.4.9 Étape I : Connexion Google Sheets via OAuth2 . . . 12
- 3.5 Sites Testés et Résultats des Tests . . . 13
- 3.6 Résultats Obtenus . . . 13
- 3.7 Difficultés Rencontrées et Solutions . . . 14

4 Conclusion Générale **14**
- 4.1 Clôture des deux exercices . . . 14
- 4.2 Leçons apprises . . . 15

1.0 Introduction

Le Web Scraping constitue une discipline essentielle de l'analyse des données modernes. Dans un contexte où d'immenses quantités d'informations sont disponibles en ligne, la capacité à extraire, transformer et stocker automatiquement ces données représente un atout majeur pour la recherche, le commerce et la prise de décision.
Ce rapport présente les travaux réalisés dans le cadre du module ***Analyse du Web***, encadré par M. Imad Faure. Le projet se compose de deux exercices complémentaires :

- **Exercice 1 :** Scraping avancé du site **Yelp** à l'aide de Python et Selenium, visant à collecter des données sur les restaurants parisiens et à analyser les profils des utilisateurs.
- **Exercice 2 :** Mise en place d'un workflow automatisé avec **n8n (via Docker)** afin d'effectuer une veille sur les prix du **Samsung Galaxy S24** sur plusieurs sites e-commerce, avec un stockage automatique des données dans Google Sheets.

Ces travaux illustrent deux approches complémentaires : le scraping programmatique avec Python, offrant un contrôle fin et une grande flexibilité, et le scraping automatisé sans code à l'aide d'outils comme n8n, permettant de concevoir rapidement des workflows efficaces.

2.0 Exercice 1 : Scraping Yelp avec Python & Selenium

2.1.0 Présentation et Objectif

Scraper les 500 meilleurs restaurants de Paris sur Yelp, extraire les informations des reviewers **(nom d'utilisateur, localisation/nationalité)**, puis établir un classement des nationalités les plus actives parmi les commentateurs.

2.2.0 Environnement Technique

Technologie	**Rôle**
Python 3	Langage de programmation principal
Selenium	Contrôle automatisé du navigateur Chrome
BeautifulSoup4	Parsing et extraction du HTML
ChromeDriver/ webdriver-manager	Interface entre Selenium et Chrome
lxml	Parser HTML performant
JSON / CSV	Formats de stockage des résultats

TABLE 1 – Environnement technique utilisé

2.3.0 Étapes Réalisées

2.3.1 Étape A : Collecte des 500 meilleurs restaurants

La première étape consiste à parcourir les pages de résultats de Yelp pour la ville de Paris, triées par note, et à extraire les informations de chaque restaurant : **nom, URL, note et nombre de reviews**.
Yelp dispose de mécanismes anti-bot sophistiqués. Pour les contourner, plusieurs stratégies ont été mises en place :

- Masquage de Selenium : injection JavaScript via le protocole CDP pour masquer toutes les traces d'automatisation (`navigator.webdriver = undefined`).
- Profil Chrome persistant : conservation des cookies et de l'historique de navigation entre les sessions.
- Pauses aléatoires : délai de 10 à 30 secondes entre chaque restaurant pour simuler un comportement humain.
- Gestion manuelle des CAPTCHAs : pause de 60 secondes sur la première page pour permettre une résolution manuelle.

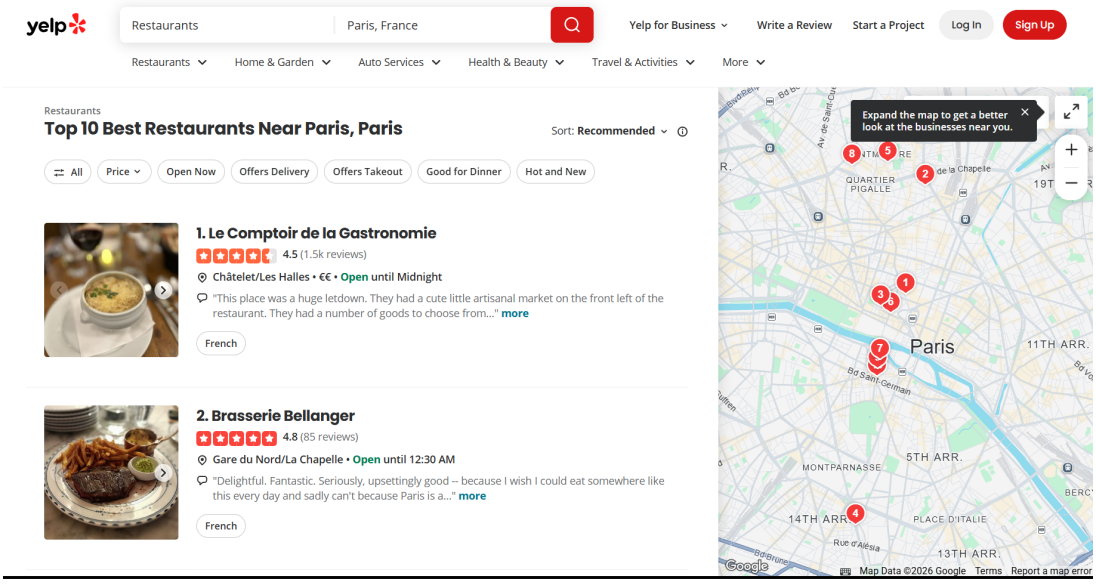


FIGURE 1 – Page de résultats Yelp (Top restaurants de Paris)

2.3.2 Étape B : Extraction des Reviewers

Pour chaque restaurant, le script parcourt toutes les pages de reviews en suivant la pagination Yelp. Les informations extraites pour chaque reviewer sont : **le nom d'utilisateur, l'URL du profil et la localisation indiquée dans le profil**.

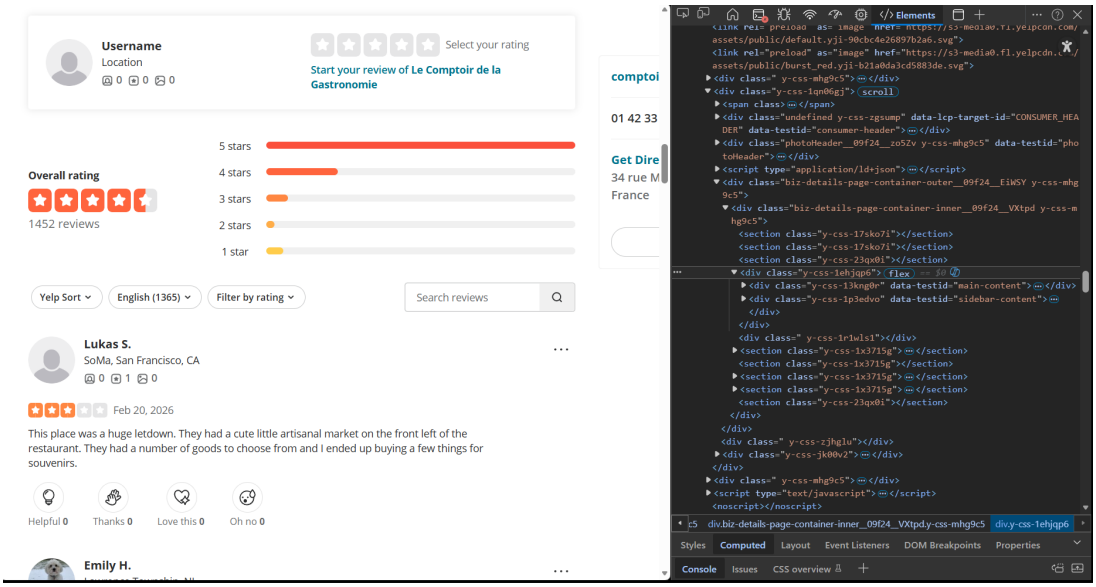


FIGURE 2 – Inspection HTML de Yelp pour identifier les sélecteurs des reviewers

2.3.3 Étape C : Extraction de la Nationalité

La nationalité est déduite de la localisation indiquée dans le profil de chaque reviewer. Le format est généralement « **Ville, Région, Pays** » **ou** « **Ville, Pays** ». Le dernier élément après la virgule est extrait comme indicateur de nationalité.

2.3.4 Étape D : Classement des Nationalités

Une fois toutes les nationalités collectées, un classement est généré par fréquence d'apparition, avec **calcul du pourcentage** par rapport au total des reviews analysées. Ce classement est exporté au format JSON.

2.4.0 Système de Checkpoint

Pour un script aussi long (500 restaurants avec plusieurs dizaines de pages de reviews chacun), un système de checkpoint a été intégré. À chaque restaurant traité, l'état complet est sauvegardé dans `output/checkpoint.json`. En cas d'interruption **(panne de courant, CAPTCHA bloquant, erreur réseau)**, le script reprend exactement là où il s'était arrêté sans perdre les données déjà collectées.

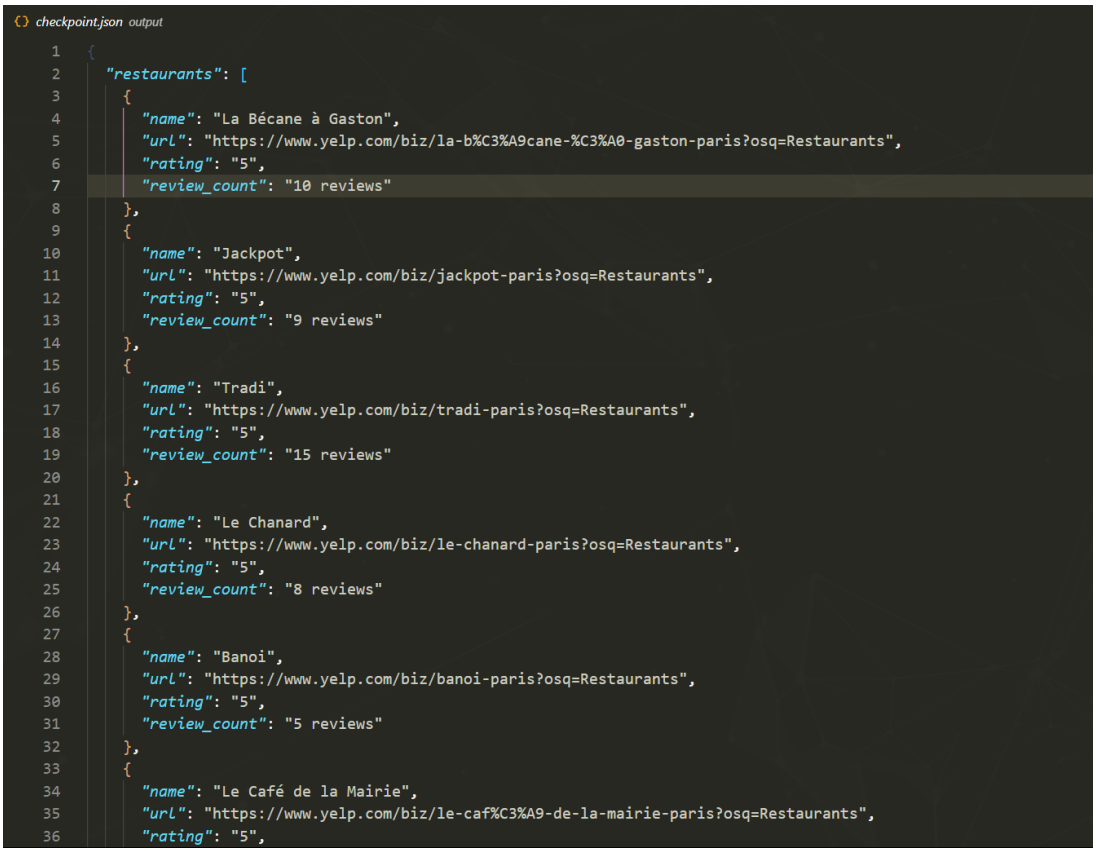


FIGURE 3 – Fichier checkpoint.json dans VS Code

2.5.0 Résultats Obtenus

À la fin du processus, les résultats suivants ont été obtenus :

- `checkpoint.json`
- `nationality_ranking.json`
- `results_reviewers.csv`

Chaque fichier contient les résultats du scraping correspondant aux données extraites du site, en cohérence avec les sélecteurs CSS utilisés et la structure HTML de Yelp.

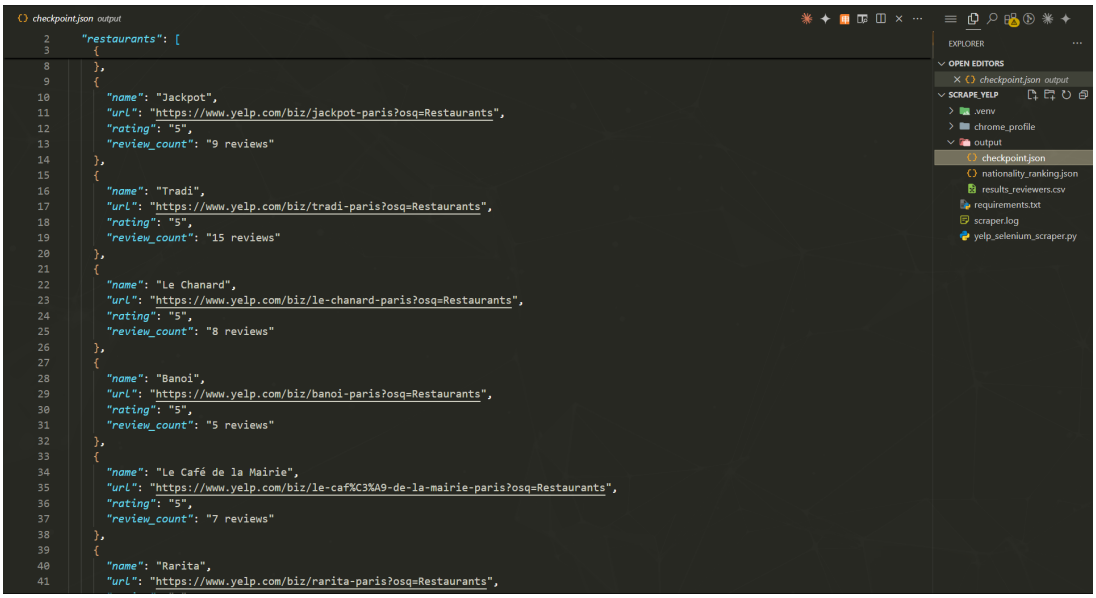


FIGURE 4 – Fichier checkpoint.json — Résultat extrait

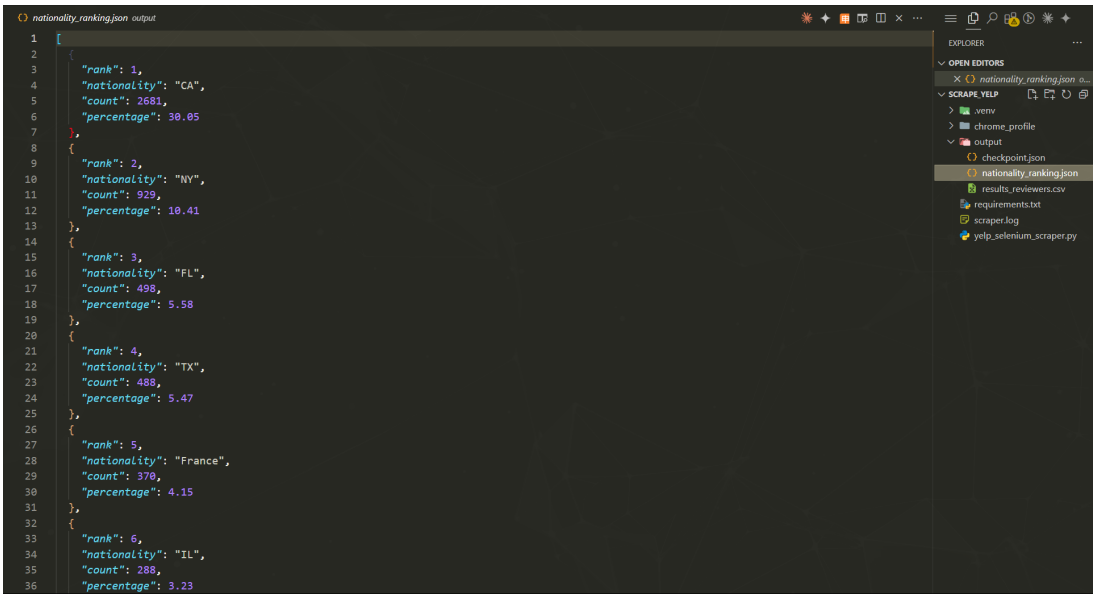


FIGURE 5 – Fichier nationality_ranking.json — Résultat extrait

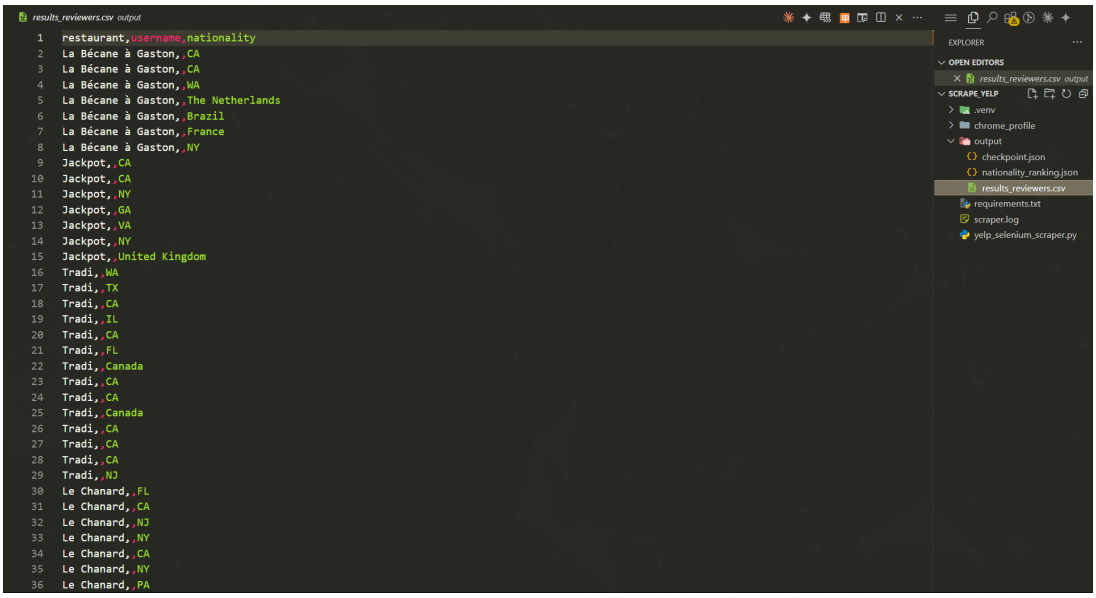


FIGURE 6 – Fichier results_reviewers.csv — Résultat extrait

2.6.0 Difficultés Rencontrées et Solutions

- **Anti-bot de Yelp :** Yelp détecte et bloque les scrapers automatiques. Solution : injection CDP pour masquer Selenium, rotation des User-Agents, profil Chrome persistant.
- **CAPTCHAs :** Son apparition sur la première page de recherche. Solution : pause de 60 secondes pour résolution manuelle avant le démarrage du scraping automatique.
- **Durée du scraping :** 500 restaurants avec plusieurs pages de reviews représentent plusieurs heures de traitement. Solution : système de checkpoint pour reprendre en cas d'interruption sans perte de données.
- **Structure HTML variable :** Yelp modifie régulièrement son HTML pour contrer les scrapers. Solution : utilisation de plusieurs sélecteurs CSS en fallback pour garantir la robustesse.

3.0 Exercice 2 : Scraping avec Workflow Automatisé avec n8n

3.1.0 Présentation et Objectif

Créer un robot de veille automatisé avec n8n qui récupère les titres et prix du produit **Samsung Galaxy S24** sur quatre sites e-commerce différents, rassemble les données dans un format uniforme et les stocke automatiquement dans Google Sheets.

3.2.0 Environnement Technique

Technologie	**Rôle**
Docker Desktop	Conteneurisation et déploiement de n8n en local
n8n	Plateforme d'automatisation de workflows (no-code / low-code)
Google Sheets	Stockage et visualisation des données scrappées
Google Cloud Console	Configuration OAuth2 pour la connexion Google Sheets
CSS Selectors	Extraction ciblée des données dans le HTML des sites

TABLE 2 – Environnement technique du workflow n8n

3.3.0 Sites e-commerce ciblés

	Site	**Lien**
1	Jumia.ma	https://www.jumia.ma
2	Boulanger.com	https://www.boulanger.com
3	AliExpress	https://www.aliexpress.com
4	Newegg.com	https://www.newegg.com

TABLE 3 – Sites e-commerce ciblés pour le scraping

3.4.0 Étapes Réalisées

3.4.1 Étape A : Installation de n8n avec Docker

n8n a été déployé via Docker en utilisant un fichier **docker-compose.yml**. Cette approche garantit un environnement isolé, portable et facile à relancer. La configuration inclut l'image officielle n8nio/n8n, l'exposition du port **5678** pour l'interface web, une authentification basique, la timezone **Africa/Casablanca** et un volume persistant pour conserver les workflows entre les redémarrages.

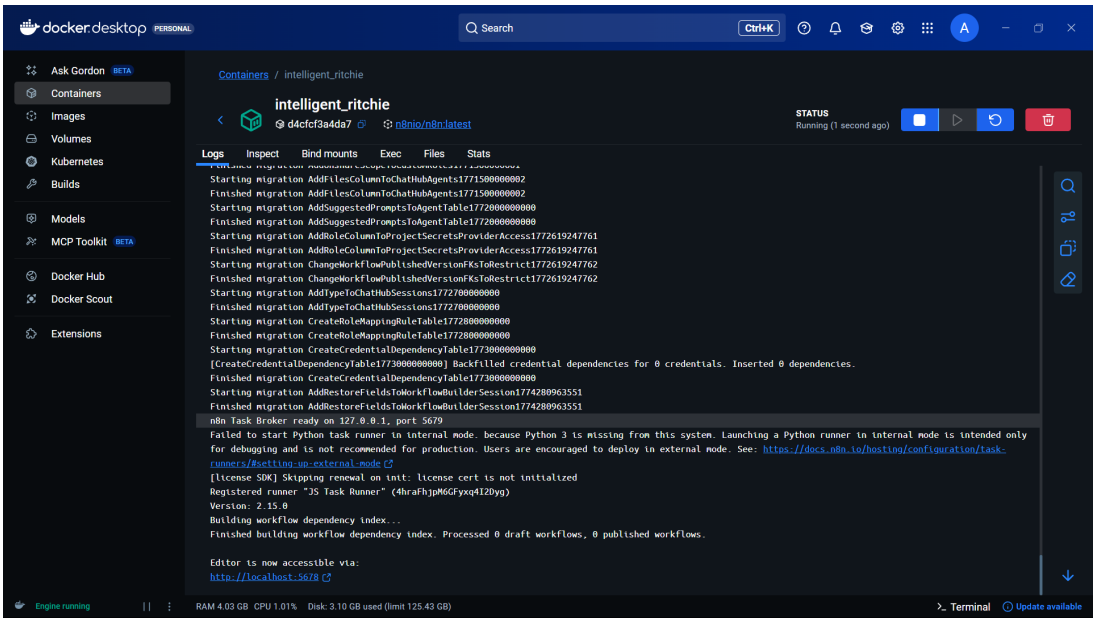


FIGURE 7 – Docker Desktop avec le conteneur n8n en cours d'exécution

3.4.2 Étape B : Accès à l'interface n8n

Après le lancement du conteneur, l'interface n8n est accessible via le navigateur à l'adresse `http://localhost:5678`. Un compte propriétaire a été créé lors du premier accès. Le workflow a été nommé **Veille-Prix-Smartphones-Multi-Sources**.

3.4.3 Étape C : Identification des sélecteurs CSS

Pour chaque site, l'outil d'inspection du navigateur a été utilisé afin d'identifier les sélecteurs CSS du titre et du prix.

3.4.4 Étape D : Création du Workflow n8n

Le workflow a été créé visuellement dans n8n en assemblant et connectant les nœuds suivants :

Nœud	Quantité	Description
Schedule Trigger	1	Déclenche le workflow automatiquement toutes les 24 heures
HTTP Request	4	Effectue une requête GET vers chaque site avec un User-Agent personnalisé
HTML Extract	4	Extrait le titre et le prix via les sélecteurs CSS identifiés
Set (Edit Fields)	4	Normalise les données : site, titre, prix, date
Merge	1	Fusionne les 4 flux de données en une seule liste (mode Append)
Google Sheets	1	Envoie chaque ligne de données dans Google Sheets

TABLE 4 – Structure du workflow n8n

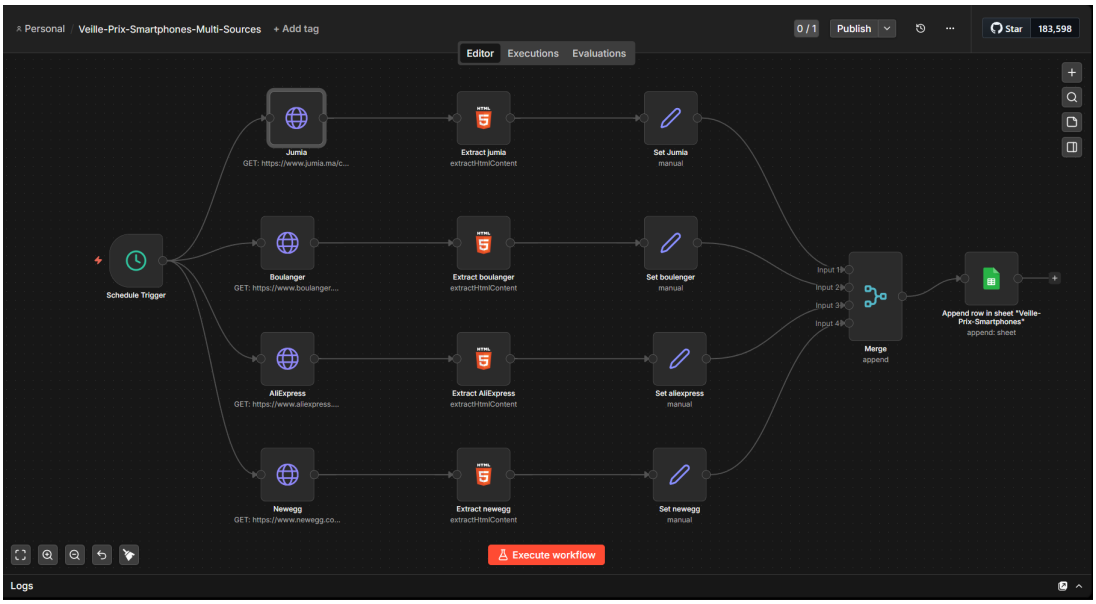


FIGURE 8 – Vue complète du workflow n8n (Veille-Prix-Smartphones-Multi-Sources)

3.4.5 Étape E : Configuration des nœuds HTTP Request

Chaque nœud HTTP Request est configuré avec la méthode **GET**, l'URL de recherche du produit sur le site concerné et un header User-Agent personnalisé **(Mozilla/5.0 Windows NT 10.0...)** afin de simuler un navigateur réel et éviter le blocage.
L'exemple suivant montre la configuration pour le site Jumia :

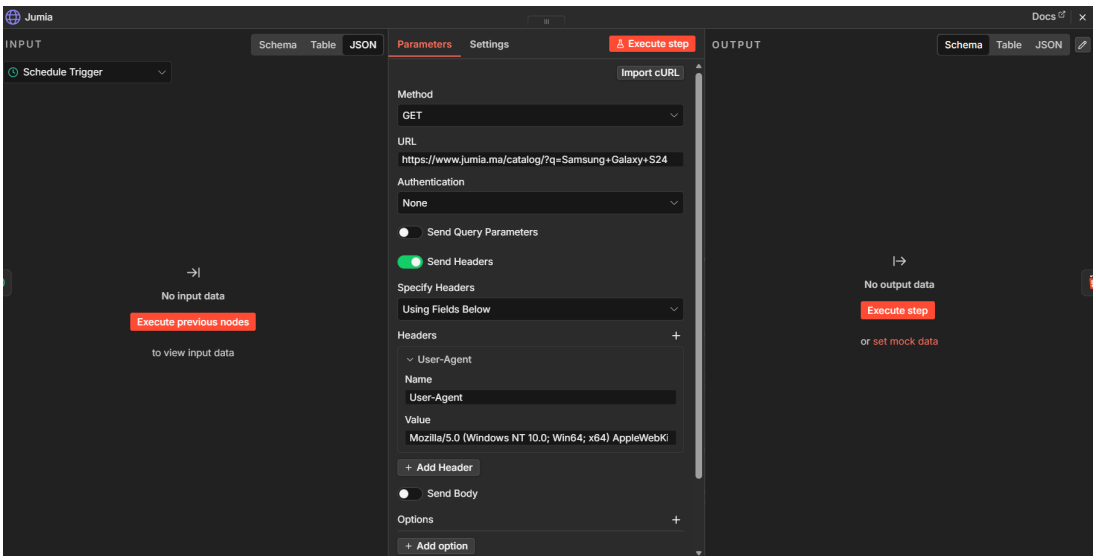


FIGURE 9 – Configuration du nœud HTTP Request (Jumia) avec sortie HTML

3.4.6 Étape F : Configuration des nœuds HTML Extract

Chaque nœud HTML Extract est configuré avec deux valeurs :

- Titre (Key : Titre, CSS Selector, Return Value : Text)
- Prix (Key : Prix, CSS Selector, Return Value : Text)

La source des données est le champ JSON `data` issu du nœud HTTP Request précédent. Exemple pour Jumia :

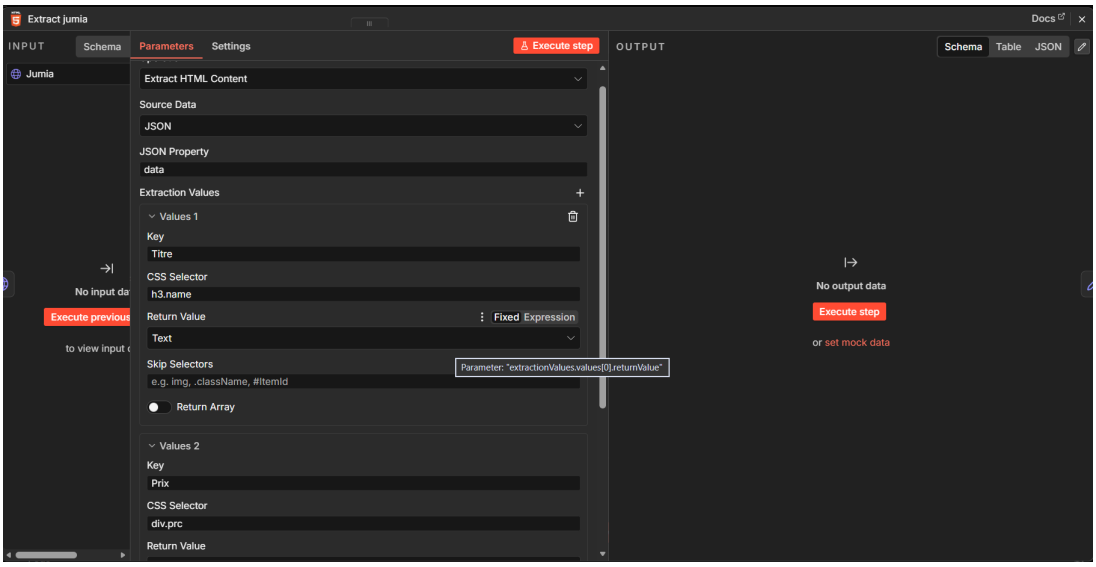


FIGURE 10 – Nœud HTML Extract avec extraction du titre et du prix (Jumia)

3.4.7 Étape G : Configuration des nœuds Set

Chaque nœud Set (Edit Fields) normalise les données dans un format uniforme avec les champs suivants :

- site : valeur fixe (ex : Jumia)
- titre : expression `{{ $json["Titre"] }}`
- prix : expression `{{ $json.Prix }}`
- date : expression `{{ $now }}`

Exemple pour Jumia :

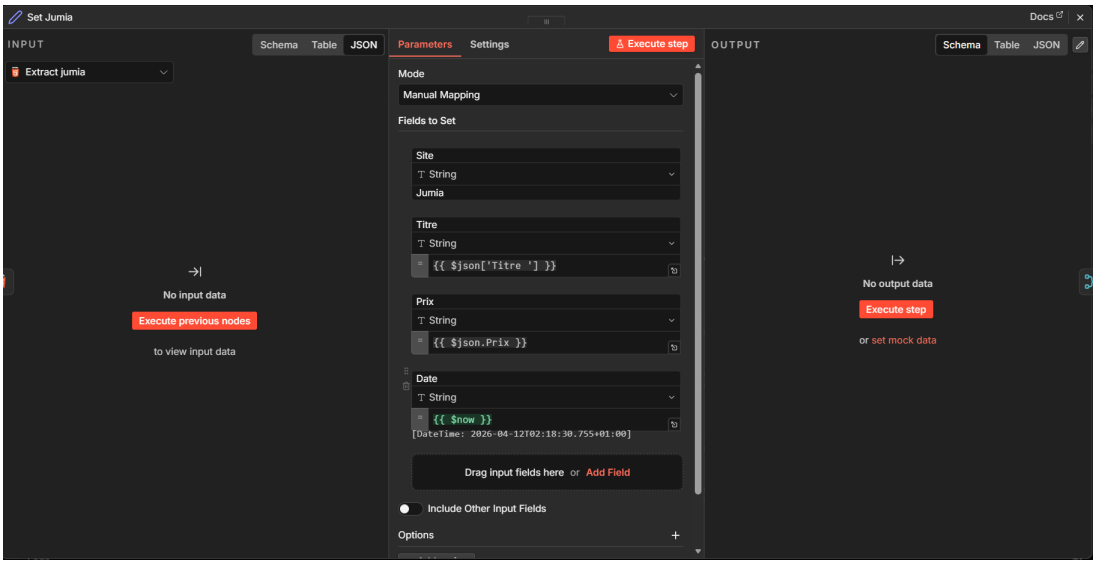


FIGURE 11 – Configuration du nœud Set (Jumia)

3.4.8 Étape H : Configuration du nœud Merge

Le nœud Merge est configuré en mode **Append** avec 4 entrées. Il reçoit les données des 4 nœuds Set (Jumia, Boulanger, AliExpress, Newegg) et les fusionne en une seule liste avant l'envoi vers Google Sheets.

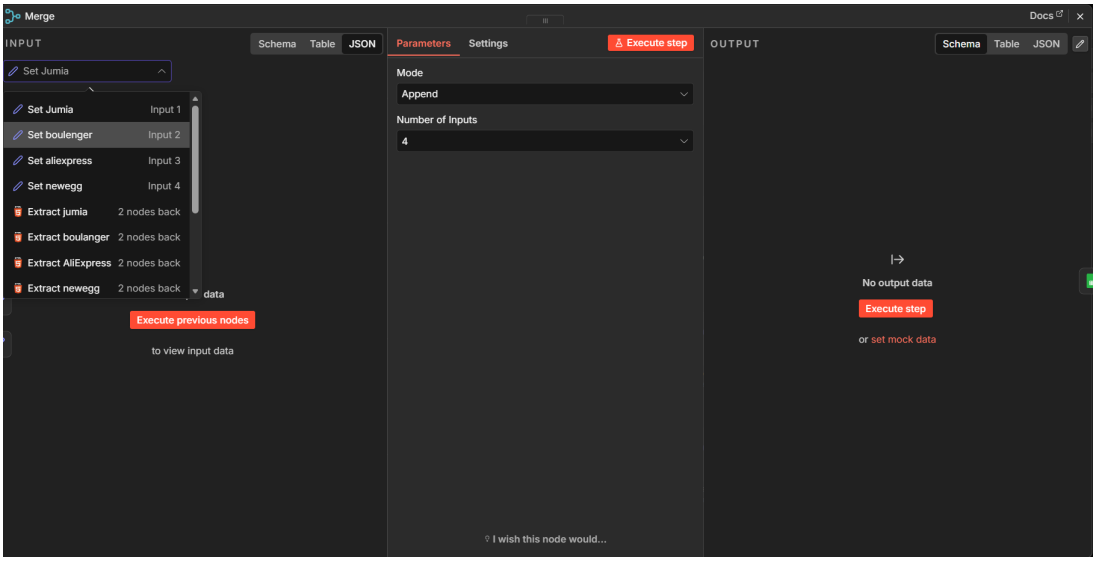


FIGURE 12 – Fusion des données des 4 sites via le nœud Merge

3.4.9 Étape I : Connexion Google Sheets via OAuth2

La connexion à Google Sheets nécessite la configuration d'une application OAuth2 sur Google Cloud Console, en suivant les étapes suivantes :

- Création d'un projet sur `console.cloud.google.com`
- Activation des API Google Sheets et Google Drive
- Configuration de l'écran de consentement OAuth (type : Externe)
- Création d'un ID client OAuth2 (type : Application Web)
- Ajout de l'URI de redirection : `http://localhost:5678/rest/oauth2-credential/callback`
- Copie du Client ID et du Client Secret dans n8n
- Connexion au compte Google via **Sign in with Google**

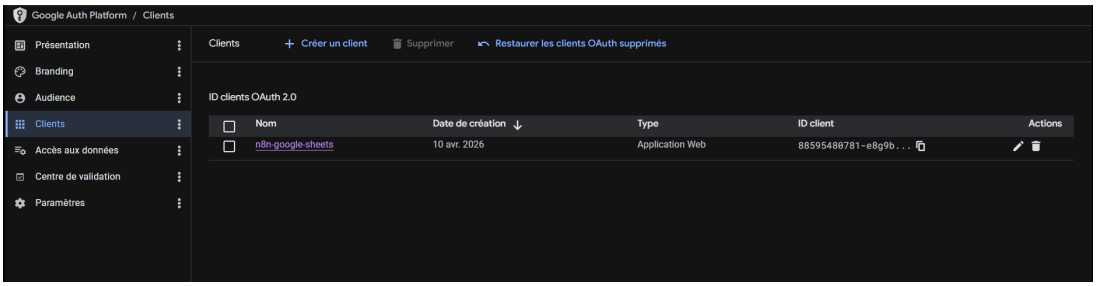


FIGURE 13 – Connexion à Google Sheets via OAuth2

3.5.0 Sites Testés et Résultats des Tests

Durant la phase de développement, de nombreux sites e-commerce ont été testés avant de retenir les quatre sites finaux. Le tableau suivant présente le bilan des tests effectués :

Site	Résultat	Raison
Jumia.ma	Succès	Retourne du HTML complet sans blocage
Boulanger.com	Succès	HTML accessible avec des sélecteurs CSS stables
AliExpress	Succès	HTML partiellement accessible avec un User-Agent personnalisé
Newegg.com	Succès	HTML complet retourné sans restriction
Amazon.fr	Bloqué	JavaScript requis / détection de bot (AWS WAF)
Alibaba.com	Bloqué	Chargement dynamique du contenu via JavaScript
Fnac.com	Bloqué	Trop de redirections
Cdiscount.com	Bloqué	Classes CSS dynamiques et contenu généré en JavaScript

TABLE 5 – Résultats des tests sur différents sites e-commerce

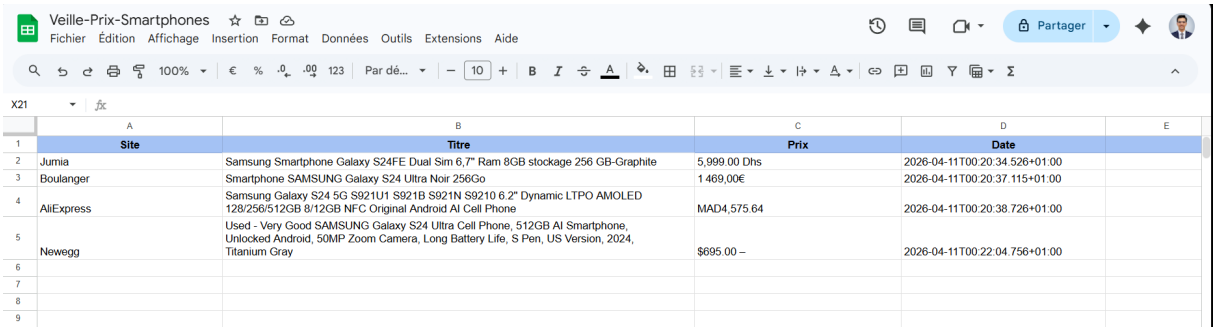
3.6.0 Résultats Obtenus

La figure ci-dessous montre l'exécution complète du workflow n8n. L'ensemble des nœuds est exécuté avec succès (indiqué en vert), ce qui valide le bon déroulement du processus automatisé de scraping et d'intégration des données dans Google Sheets.



FIGURE 14 – Workflow n8n après exécution (tous les nœuds en succès)

Données stockées dans Google Sheets :
Lien vers Google Sheets: https://docs.google.com/spreadsheets/...



	Site	Titre	Prix	Date
2	Jumia	Samsung Smartphone Galaxy S24FE Dual Sim 6,7" Ram 8GB stockage 256 GB-Graphite	5,999.00 Dhs	2026-04-11T00:20:34.526+01:00
3	Boulanger	Smartphone SAMSUNG Galaxy S24 Ultra Noir 256Go	1 469,00€	2026-04-11T00:20:37.115+01:00
4	AliExpress	Samsung Galaxy S24 5G S921U1 S921B S921N S9210 6.2" Dynamic LTPO AMOLED 128/256/512GB 8/12GB NFC Original Android AI Cell Phone	MAD4,575.64	2026-04-11T00:20:38.726+01:00
5	Newegg	Used - Very Good SAMSUNG Galaxy S24 Ultra Cell Phone, 512GB AI Smartphone, Unlocked Android, 50MP Zoom Camera, Long Battery Life, S Pen, US Version, 2024, Titanium Gray	$695.00 –	2026-04-11T00:22:04.756+01:00

FIGURE 15 – Google Sheets contenant les données collectées à partir des 4 sites

3.7.0 Difficultés Rencontrées et Solutions

- **Blocage des grands sites :** Amazon, eBay, Fnac, Alibaba et Cdiscount bloquent les requêtes HTTP simples, car leur contenu est généré dynamiquement via JavaScript. **Solution :** sélection de sites retournant du HTML statique complet (Jumia, Boulanger, AliExpress, Newegg).
- **Configuration OAuth2 complexe :** La connexion à Google Sheets nécessite la création et la configuration d'une application sur Google Cloud Console, ce qui constitue une procédure relativement complexe pour les débutants. **Solution :** suivi détaillé des étapes de configuration (création du projet, activation des API, configuration OAuth2) et utilisation de la documentation officielle.
- **Variabilité des structures HTML :** Chaque site possède une structure HTML différente, ce qui complique l'extraction uniforme des données. **Solution :** adaptation des sélecteurs CSS pour chaque site et normalisation des données via les nœuds `Set`.

4.0 Conclusion Générale

4.1.0 Clôture des deux exercices

Ce projet nous a permis d'explorer et de comparer deux approches complémentaires du **Web Scraping**, chacune présentant ses avantages et ses limites.

- **L'Exercice 1 (Python / Selenium / Yelp)** illustre une approche avancée du scraping basée sur la programmation. Cette méthode offre une grande flexibilité et permet de traiter des cas complexes tels que les CAPTCHAs, les sites dynamiques utilisant JavaScript ainsi que les mécanismes anti-bot sophistiqués. Elle est particulièrement adaptée aux projets nécessitant un contrôle fin et une personnalisation avancée du processus de collecte.
- **L'Exercice 2 (n8n / Docker / Google Sheets)** met en évidence l'efficacité des outils d'automatisation no-code pour la mise en place rapide de solutions de veille. Sans recourir à un développement intensif, il a été possible de concevoir un pipeline complet permettant de collecter, transformer et stocker automatiquement des données issues de plusieurs sites e-commerce.

4.2.0 Leçons apprises

- Les grands sites e-commerce (Amazon, eBay, Fnac) investissent fortement dans des mécanismes de protection anti-scraping, notamment via le chargement dynamique en JavaScript et des systèmes de sécurité tels que les Web Application Firewalls (WAF).
- Le scraping éthique et responsable nécessite le respect des fichiers `robots.txt` ainsi que des conditions d'utilisation des sites web.
- Les outils d'automatisation comme n8n permettent de démocratiser le scraping en le rendant accessible sans compétences avancées en programmation.
- La mise en place de systèmes de checkpoint est essentielle pour garantir la robustesse et la continuité des processus de collecte de données sur le long terme.

En conclusion, ce projet met en évidence l'importance de choisir l'outil et l'approche adaptés en fonction du contexte, du niveau de complexité du site cible et des objectifs du projet. La combinaison des approches programmatiques et no-code constitue une stratégie efficace pour répondre à des besoins variés en extraction et exploitation de données web.